Unit 2 - Summarizing Data – Numerical Methods[1]

Measures of Center, Variation, and Location

Measures of Center

As a computing professional, you'll encounter data constantly: API response times, user engagement metrics, model training losses, server loads, A/B test results, and more. Understanding central tendency helps you answer questions like "What's typical?" or "What should I expect?"

The Mean (Arithmetic Average)

Definition: The sum of all values divided by the number of values.

Formula: $\boxed{\bar{x} = \frac{(\Sigma x)}{n}}$ n: sample size

When to use it: The mean is your default choice for symmetric distributions without extreme outliers.

Real-world computing examples:

- **Average API latency:** If your e-commerce API responds in 45ms, 50ms, 48ms, 52ms, and 47ms, the mean latency is 48.4ms. This helps you set SLA targets.
- **Mean training loss:** In machine learning, you track mean loss across batches to monitor if your model is learning. A decreasing mean loss over epochs indicates progress.
- **Average session duration:** For a streaming app, if users watch for 22, 35, 28, 31, and 29 minutes, the mean is 29 minutes—useful for infrastructure planning.

class	f_i	Mid
[0, 5)	10	2.5
[5, 10)	8	7.5
[10, 15)	2	12.5

$$\bar{x} = \frac{(10 \times 2.5 + 8 \times 7.5 + 2 \times 12.5)}{(10 + 8 + 2)} = 5.5$$

Weakness: Extremely sensitive to outliers. If one API call takes 5000ms due to a network hiccup, your mean skyrockets and no longer represents "typical" performance.

The Median

Definition: The middle value when data is sorted. If there's an even number of values, it's the average of the two middle values.

When to use it: Perfect for skewed distributions or when outliers are present.

Computing examples:

- **Median response time:** If 99% of your requests complete in 50ms but 1% take 10 seconds due to database locks, the median (~50ms) better represents typical user experience than the mean (~150ms).

[1] I have used Claude Sonnet 4.5 to edit, structure and generate some of the examples in these lecture notes

- **Median salary data:** When analyzing developer salaries, the median is more informative than the mean because extreme high earners skew the average upward.
- **Median file size:** In a file storage system, if most files are 2-5MB but a few videos are 2GB, the median tells you what typical users are uploading.

Example:

$response_times$ = c(45, 50, 48, 5000, 47) # one outlier

$median_time = median(response_times)$ # Returns 48, not affected by 5000

Sort low to high: 45, 47, 48, 50, 5000 Median = 48 ms Median is not sensitive to outliers.

Companies like Netflix and Amazon track median (p50) metrics alongside p95 and p99 percentiles for performance monitoring.

The Mode

Definition: The most frequently occurring value(s) in a dataset.

When to use it: Categorical data, discrete values, or understanding the most common occurrence.

Computing examples:

- **Most common error code:** If your logs show HTTP status codes [200, 200, 200, 404, 200, 500, 200], the mode is 200 (success), but the presence of 404 and 500 matters for debugging.
- **Most popular screen resolution:** Analytics might show that 1920×1080 is the modal resolution, informing your responsive design decisions.
- **Common user actions:** In a mobile app, if "scroll" appears 1000 times, "tap" 500 times, and "swipe" 200 times, the mode is "scroll."

Example:

$error_codes$ = c(200, 200, 200, 404, 200, 500, 200)

$mode_result$ = names(sort(table(error_codes), decreasing = TRUE))[1]

Returns "200"

Special cases: Bimodal (two modes) or multimodal distributions are common in computing. For example, response times might cluster around 50ms (cache hits) and 200ms (database queries), revealing two distinct behaviors.

No mode, one mode, two mode (bimodal), more modes (multimodal) these are possible

Comparing the Three:

Let's say you're analyzing daily active users (DAU) for a social media feature over two weeks:

Dataset: [1200, 1250, 1180, 1300, 1220, 15000, 1190, 1240, 1210, 1230, 1200, 1180, 1260, 1220]

- **Mean:** ~2,562 DAU (misleading due to the viral spike on day 6)
- **Median:** 1,220 DAU (represents typical daily engagement)
- **Mode:** 1,200 DAU (most common engagement level)

Key takeaway: For deployment decisions, you'd trust the median here. The mean suggests you need 2.5x infrastructure, when actually the spike was an anomaly.

Measures of Variation

Two systems can have the same average performance but wildly different reliability. Consider two APIs both averaging 100ms response time:

- **API A:** Always returns in 95-105ms (low variation, predictable)
- **API B:** Returns in 10ms or 500ms unpredictably (high variation, same mean)

Which would you choose for production? Variation metrics tell you about consistency, reliability, and risk.

Range

Definition: Maximum value minus minimum value.

Formula: Range = max(x) - min(x)

Example: If your fastest query takes 20ms and slowest takes 2000ms, range = 1980ms. This tells you the worst-case scenario.

Limitation: Only considers two values (extremes), ignoring everything in between. Not robust to outliers.

Interquartile Range (IQR)

Definition: The range of the middle 50% of data (Q3 - Q1).

Formula: IQR = Q3 - Q1

Where Q1 is the 25th percentile and Q3 is the 75th percentile.

Why it's better: Resistant to outliers because it ignores the extreme 25% on each end.

Computing examples:

- **Performance monitoring:** If Q1 = 45ms and Q3 = 55ms, your IQR = 10ms. This means 50% of requests complete within a 10ms window—very consistent performance.
- **Outlier detection:** Values below Q1 - 1.5×IQR or above Q3 + 1.5×IQR are typically flagged as outliers. This is how box plots identify anomalies in monitoring dashboards.

Modern relevance: Cloud monitoring tools (DataDog, Grafana, New Relic) use IQR-based alerting to detect anomalies automatically.

Variance

Definition: The average of squared deviations from the mean.

Formula (population): $\sigma^2 = \frac{\Sigma(x-\mu)^2}{N}$ //not used in statistical analysis, only hypothesized

Formula (sample): $\boxed{s^2 = \frac{\Sigma(x-\bar{x})^2}{n-1}}$ //Actually used

Intuition: Variance measures how far data points spread from the mean on average. Squaring ensures negative and positive deviations don't cancel out, and it heavily penalizes outliers.

Note on units: Variance is in squared units (ms^2), which is hard to interpret. That's why we usually use standard deviation instead.

Standard Deviation

Definition: The square root of variance.

unit of Sd : Same as data unit

Sd = 0 means

Formula (population): $\sigma = \sqrt{\sigma^2}$

Formula (sample): $s = \sqrt{s^2}$

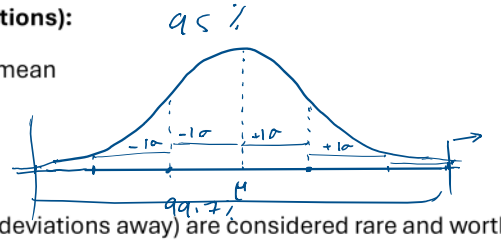
Why it's essential: Standard deviation is in the same units as your data, making it interpretable. "Response times have a standard deviation of 15ms" is meaningful.

Computing examples:

- **SLA definition:** You might guarantee "95% of requests complete within mean $\pm$ 2×SD" based on the empirical rule (for normal distributions, ~95% fall within 2 SDs).
- **Model evaluation:** In ML, you report mean accuracy ± SD across cross-validation folds. Example: "Accuracy: 94.2% ± 1.3%" tells readers about both performance and consistency.
- **Quality control:** If your ETL pipeline processes files in 120 ± 5 minutes (mean ± SD), that's predictable. But 120 ± 45 minutes signals instability requiring investigation.

The Empirical Rule (for normal distributions):

- ~68% of data falls within 1 SD of mean
- ~95% within 2 SDs
- ~99.7% within 3 SDs



This is why "3-sigma events" (3 standard deviations away) are considered rare and worthy of investigation.

Coefficient of Variation (CV)

Definition: Standard deviation divided by the mean, expressed as a percentage.

Formula: $CV = \left(\frac{S}{\bar{x}}\right) \times 100\%$

Use CV for comparison purposes (if the two data sets are on different scales/units)

When to use it: Comparing variability across datasets with different units or scales.

Computing examples:

- **Comparing microservices:** Service A averages 50ms ± 5ms (CV = 10%). Service B averages 500ms ± 25ms (CV = 5%). Despite higher absolute variation, Service B is more consistent relative to its mean.
- **Resource utilization:** Comparing CPU usage (%) vs memory usage (GB) across servers requires CV, not raw SD.

R implementation:

```
cv <- (sd(response_times) / mean(response_times)) * 100
```

Example: Let's say you measured API response times (in milliseconds) for 5 requests:

Dataset: 45, 50, 48, 52, 47

Calculate the coefficient of variation by manual calculations:

$$n = 5$$

$$\bar{x} = \frac{45 + 50 + 48 + 52 + 47}{5} = \frac{242}{5} = 48.4\ ms$$

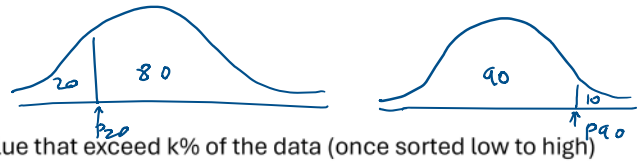
x	$(x - \bar{x})$	$(x - \bar{x})^2$
45	-3.4	11.56
50	1.6	2.56
48	-0.4	0.16
52	3.6	12.96
47	-1.4	1.96

$$\sum(x - \bar{x})^2 = 29.2 \qquad S^2 = \frac{29.2}{4} = 7.3 \qquad S = \sqrt{7.3} = 2.7\ ms$$

$$CV = \frac{2.7}{48.4} \times 100\% = 5.5\%$$

Measures of Location

Percentiles and Quantiles



Definition: The kth percentile is the value that exceed k% of the data (once sorted low to high)

20th

examples:

- **SRE golden metrics:** Google's Site Reliability Engineering emphasizes p50, p95, and p99 latencies. Why? Because p99 captures the worst 1% of user experiences, which can still affect millions of users at scale.
- **Capacity planning:** If p95 response time is 200ms, you know 95% of users experience sub-200ms latency. This informs infrastructure decisions.
- **Performance budgets:** Modern web development sets budgets like "p75 page load time < 3 seconds" to ensure most users have fast experiences.

R implementation:

Probs = c(0.50, 0.95, 0.99)

p50 = quantile(response_times, 0.50) # median

Manual calculation:

Sort data in ascending order. Index them from 1 to n

Find the locator index: $l = n \times \frac{k}{100}$

If l is an integer, $P_k = \frac{x_l + x_{l+1}}{2}$ else, round up l and report the value in that location: $P_k = x_{[l]}$

Example:

Your CDN has these response times (ms) for 20 requests:

[12, 15, 18, 20, 22, 25, 28, 30, 32, 35, 38, 40, 45, 50, 55, 60, 70, 85, 100, 200]

L = 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20

Calculate p50, p75, p90, p95, and p99. If you wanted to guarantee that 95% of users experience sub-100ms latency, are you meeting that goal?

P_{50} $\quad l = 20 \times \frac{50}{100} = 10 \qquad Med = P_{50} = \frac{35+38}{2} = 36.5$ ms

P_{90} $\quad l = 20 \times \frac{90}{100} = 18 \qquad P_{90} = \frac{85+100}{2} = 92.5$ ms

Only 10% of CDN response times exceed 92.5 ms
90% the times, the CDN response times are below 92.5 ms

P_{95}: $\quad l = 20 \times \frac{95}{100} = 19 \qquad P_{95}: \frac{100+200}{2} = 150\ ms$

P_{99}: $\quad l = 20 \times \frac{99}{100} = 19.8\ round\ up: 20 \quad P_{99} = 200\ ms$

$$z_{score} = \frac{x - \bar{x}}{s} = -1$$

Z-Scores (Standard Scores)

Definition: The number of standard deviations a value is away from the mean.

Formula: $z = \frac{(x - \bar{x})}{s}$ $z_{score} = -1$: our score is one standard deviation below the average

When to use it: Identifying outliers, comparing values from different distributions, or standardizing data for using known distributions.

Computing examples:

- **Anomaly detection:** If API response times normally follow a distribution with mean 100ms and SD 10ms, a response time of 150ms has z = 5. That's 5 standard deviations away—highly unusual and worth investigating. $z = \frac{150 - 100}{10} = \frac{50}{10} = 5$
- **Feature scaling:** Before training neural networks, you standardize features to have mean 0 and SD 1 using z-scores. This prevents features with larger scales from dominating gradient descent.
- **Comparing across metrics:** You can't directly compare "200ms latency" to "85% CPU usage," but you can compare their z-scores to see which is more anomalous.

R implementation:

Mean(Z-scores) = 0

Sd (Z-scores) = 1

```
# Calculate z-scores

z_scores <- scale(response_times)  # Returns a matrix

z_scores <- as.vector(z_scores)   # Convert to vector
```

Interpretation:

$|z| > 3$

- z = 0: Value equals the mean
- z = 1: One SD above mean
- z = -2: Two SDs below mean
- |z| > 3: Likely an outlier (in normal distributions)

Common Pitfalls to Avoid

1. **Using mean when median is appropriate:** Always check for skewness first
2. **Ignoring variance:** Two systems with same mean can have very different reliability
3. **Forgetting about outliers:** They might be bugs, not features
4. **Sample size matters:** Small samples make statistics unreliable (n < 30 is risky)
5. **Simpson's Paradox:** Aggregated data can show opposite trends to subgroups

In practice: Always report multiple statistics. A single number (like mean) rarely tells the full story. Combine center + variation + location for complete understanding and always visualize your data before making decisions.

Quick Reference Cheat Sheet

Metric	When to Use	R Code
Mean	Symmetric data, no outliers	mean(data)
Median	Skewed data, outliers present	median(data)
Mode	Categorical or discrete data	Custom function
Range	Quick spread check	max(data) - min(data)
IQR	Outlier-resistant spread	IQR(data)
Std Dev	Quantifying variability	sd(data)
CV	Comparing different scales or units	(sd(data)/mean(data)) * 100
Percentiles	Understanding distribution shape	quantile(data, probs = c(...))
Z-Score	Outlier detection, standardization	scale(data) or (x - mean(x)) / sd(x)